28. b. Singly Linked List – TraverseListReverseOrderRecursive – Space Complexity and Algorithm.

***Program*: –**

```
void TraverseListReverseOrderRecursive(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {

        if ((*head)->next != nullptr)
        {
            TraverseListReverseOrderRecursive(&((*head)->next));
        }
        cout << (*head)->data << " ";
    }
}
….
 case 26:
        {
            cout << endl;
            cout << endl;
            cout << "Traverse List Reverse Order Recursive" << endl;
            TraverseListReverseOrderRecursive(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

Space Complexity Analysis

1. *Input Space Complexity*

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, O(1).***

Thus, the total input space complexity is: O(1).

• One argument: Node **head – a single pointer-to-pointer (one machine word).

The amount of memory needed for the formal parameter is fixed and independent of the list size.

This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

B. Full – data (inclusive) Definition:

• Everything reachable from the input pointer when the call starts must be counted.

• *head may reference a singly-linked list containing n nodes.

• Those n Node objects dominate ⇒ total input footprint Θ(n).

This more common definition includes the entire data structure. If the linked list has **`n` nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

2. *Auxiliary Space Complexity*

Auxiliary space complexity refers to the extra space used by the algorithm that is not part of the input. In the case of the TraverseListReverseOrderRecursive function, the auxiliary space is determined by the variables declared within the function:

What the algorithm itself allocates in addition to its input:

• Recursion stack

- Each call contributes one stack frame that holds:

  – the saved return address,

  – the single parameter (OBJ),

  – a few compiler-generated temporaries.

- Size per frame is O(1).

- Depth of recursion = number of nodes visited = n (worst case).

⇒ Stack space = n × O(1) = O(n).

• No dynamic heap allocations and no additional data structures are created inside the function.

Therefore the auxiliary (working) space consumed by the routine is linear in the length of the list.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For a recursive function, the primary source of auxiliary space is the **call stack**.

1. **Call Stack Usage**: Each time the function calls itself on the `next` node, the system creates a "stack frame" on the call stack. This frame stores the function's parameter (`head`) and the return address. The size of each frame is constant.
2. **Recursion Depth**: The function will continue to call itself until it reaches the end of the list. For a list with `n` nodes, this results in a chain of `n` nested function calls before the base case is met and the `cout` statements begin to execute.
3. **Total Space**: The total auxiliary space is the size of one stack frame (which is constant, O(1)) multiplied by the maximum depth of the recursion (which is `n`).

The space required by the call stack grows linearly with the number of nodes in the list.

Therefore, the auxiliary space complexity is **Θ(n)**.

$Algorithm$

$TraverseListReverseOrderRecursive(START)$

$1.\ [START\ is\ NULLPTR?\ ]if\ START = NULL, then:$

$Write:``\ Error: List\ is\ Empty.``$

$Return\ and\ exit.$

$Else$

$if\ Link[START] \neq NULL, then:$

$TraverseListReverseOrderRecursive(ADDRESS\ of\ LINK[START])$

$- -$

$[\ Translation\ of\ PUSH\ Operation\ in\ ``TraverseListReverseOrderRecursive(ADDRESS\ of\ LINK[START])``]$

$A.PUSH$

$a.\ [Push\ current\ values\ \ onto\ stacks.]$

$(i)\ Set\ TOP := TOP + 1$

$(ii)\ Set\ STACK[TOP] := ADDRESS\ of\ LINK[START]$

$b.[Reset\ parameters]$

$Get\ START := \quad ADDRESS\ of\ LINK[START]$

$c.Go\ to\ Step\ 1.$

$- -$

$[\ Translation\ of\ POP\ operation\ in\ ``\ TraverseListReverseOrderRecursive(ADDRESS\ of\ LINK[START])``]$

$B.POP$

$a.\ [POP\ the\ TOP\ Stack.]$

$(i)\ Set\ TOP := TOP - 1$

$b.[Reset\ parameters]$

$Get\ START := \quad STACK[TOP]$

$[When\ POP\ in\ STACK\ sets\ TOP\ to\ NULL]$

$(a) if\ TOP := NULL, then:$

$(b)\ \ [Restore\ top\ values\ on\ stack]$

$(i)\ Set\ START := STACK[TOP]$

$(ii)\ Set\ TOP := TOP - 1$

$(c)\ Exit$

$- -$

$Write: INFO[START]$

[***END OF IF STRUCTURE***]

[***END OF ALGORITHM***]